

Introduction

Dans ce premier chapitre, nous nous intéresserons tout d'abord au cadre général de notre projet et la présentation de l'organisme d'accueil. Nous abordons après l'étude de l'existant et nous exposerons ensuite la solution proposée. Enfin, nous terminerons ce chapitre par énoncer la méthodologie de gestion adoptée.

1.1 Contexte du projet

Dans le cadre de la transformation numérique, les entreprises modernisent leurs infrastructures informatiques pour répondre aux exigences croissantes en matière de flexibilité, de scalabilité et de coût. Le cloud computing est devenu une solution incontournable pour ces entreprises, leur offrant la possibilité de déployer et de gérer des ressources informatiques de manière dynamique et optimisée. Cependant, la gestion des infrastructures cloud peut s'avérer complexe, nécessitant une expertise technique avancée ainsi Les approches traditionnelles de gestion des infrastructures informatiques, souvent caractérisées par des déploiements manuels et une faible automatisation, ne suffisent plus pour répondre aux exigences croissantes en matière de rapidité, de scalabilité . Pour simplifier ce processus, des outils d'infrastructure as code (IaC) comme Terraform ont été développés, facilitant ainsi la création, la mise à jour et la gestion des ressources cloud de manière automatisée. Malgré ses avantages, la rédaction manuelle des scripts Terraform peut être lente et sujette aux erreurs, surtout pour les utilisateurs novices ou ceux qui gèrent des architectures cloud complexes. C'est dans ce contexte qu'intervient notre solution : une application innovante qui s'appuie sur des outils comme Terraform, Docker et des bibliothèques graphiques et ambitionne de démocratiser la gestion des infrastructures informatiques .

1.2 Présentation de l'organisme d'accueil

Dans ce qui suit nous présentons l'organisme d'accueil dans laquelle s'est effectué notre stage de fin d'études, son secteur d'activité et ses services .

1.2.1 Trend-IT

TREND-IT est une entreprise française spécialisée dans les technologies Microsoft, offrant des services de conseil et d'expertise technique pour soutenir les projets de transformation digitale.

Avec une expertise reconnue dans le développement d'applications, la Business Intelligence, DevOps et l'agilité, TREND-IT aide ses clients à naviguer dans le monde complexe de la technologie pour optimiser leurs opérations et atteindre leurs objectifs commerciaux. Fondée en 2017, l'entreprise compte 25 consultants techniques et fonctionnels, offrant une assistance étroite et personnalisée à ses clients dans leur quête de modernisation technologique.

1.2.2 Secteurs d'activités

L'entreprise « Trend-IT » possède un ensemble d'applications dans le but de satisfaire les besoins de ses clients. En effet, elle favorise l'exploitation fructueuse des nouvelles technologies et essaye de répondre aux exigences de marché avec ces experts en architecture Web, Mobile Applicative de qualité ,Devops et Bussiness intelligence. nous pouvons présenter une des solutions implémentées au sein du Trend-IT.

- **Trend-It Worklog** : WorkLog est une application web puissante conçue pour aider les professionnels à gérer et suivre leur temps de travail de manière efficace.

1.3 Présentation du projet

Ce projet vise à créer une application de génération du script terraform des ressources cloud à partir d'une interface graphique "Trend-IT InfraGen" pour arriver à satisfaire les besoins mentionnés et les problématiques présentés.

Tout d'abord, nous avons présenté le contexte du projet et maintenant nous allons réaliser une étude de l'existant afin de diagnostiquer les d'une part. Nous allons ainsi exposer le travail demandé, dans un deuxième temps.

1.3.1 Etude de l'existant

Au sein de notre organisme les expert Cloud et Devops gèrent les infrastructures Cloud d'une manière traditionnelle et manuelle, après une analyse des solutions concurrents,nous avons trouvé des outils de génération de diagrammes d'infrastructure cloud, ces derniers se concentrent principalement sur l'aspect graphique.D'autres solutions comme Microsoft Azure permettent de créer manuellement des services Cloud mais un seul à la fois, ce qui ne facilite pas la création d'une architecture complète ou de modèles intégrés.D'autres outils de visualisation et de génération de diagrammes d'infrastructure cloud ont été identifiés, chacun avec ses propres caractéristiques et

limitations. Par exemple "Cloudcraft" se distingue par sa capacité à visualiser des diagrammes AWS de manière graphique à partir d'une architecture déjà existante sur Azure. Cependant, cette solution est limitée et ne sert pas à créer une nouvelle architecture. "Brainboard", se positionne comme la seule option en marché capable de générer des scripts Terraform à travers des présentations graphiques des ressources, mais elle présente des contraintes en termes de coût et vise à améliorer l'expérience graphique. Bien que ces solutions offrent des fonctionnalités intéressantes pour la visualisation et la création de diagrammes cloud, elles présentent des limites en termes de flexibilité, de coût et de capacité à gérer des architectures complexes, alors voici brièvement les principales problématiques que nous avons identifiées :

- **Complexité de la gestion des ressources cloud** : La multiplicité des composants et la diversité des configurations rendent la gestion des ressources cloud difficile et sujette à des erreurs.
- **Manque d'automatisation** : L'absence d'outils automatisés pour la configuration et le déploiement des infrastructures augmente le risque d'erreurs et ralentit les processus.
- **Non-reproductibilité des configurations** : Les configurations manuelles ne garantissent pas la reproductibilité, ce qui peut entraîner des incohérences entre les environnements de développement, de test et de production.
- **Sécurité et conformité** : La gestion manuelle des configurations peut compromettre la sécurité et la conformité aux meilleures pratiques et aux réglementations en vigueur.

Le projet TREND-IT InfraGen vise à résoudre ces problématiques en développant une application web permettant la génération automatique des scripts Terraform pour des ressources cloud en utilisant une interface graphique conviviale.

1.3.2 Solution proposée

Pour remédier aux limitations de génération d'infrastructure existante, nous proposons de mettre en place une application web innovante conçue pour simplifier et automatiser la génération de scripts Terraform à partir des ressources cloud. Cette solution offre plusieurs avantages :

- **Simplification et Automatisation** : Réduction de la complexité et de la charge de travail associées à la gestion des infrastructures cloud grâce à l'automatisation.

- **Cohérence et Reproductibilité** : Assurer des déploiements cohérents et reproductibles, réduisant les risques d'erreurs et d'incohérences.
- **Sécurité et Conformité** : Faciliter l'application des politiques de sécurité et de conformité grâce à des configurations automatisées et vérifiées.
- **Efficacité et Performance** : Accélérer les cycles de déploiement et améliorer la performance globale des processus de gestion des infrastructures.

1.4 Méthodologies et formalisme adoptés

Dans cette section nous allons commencer, dans un premier temps, par présenter la méthodologie de gestion de projet adoptée. Nous allons exposer le formalisme que nous avons opté pour la conception, dans un deuxième temps.

1.4.1 Méthodologie de gestion de projet

Le choix entre un modèle de processus de développement et un autre, dépend de la nature du projet et sa taille. Lorsqu'il s'agit d'un projet où les données ne sont pas réunies dès le départ, où les besoins sont incomplets, il est recommandé de s'orienter vers une méthode itérative ou orientée prototypes.

1.4.1.1 Les méthodologies Agiles

Parmi les méthodes itératives largement utilisées de nos jours, nous pouvons citer la méthodologie "Agile". Certes, c'est une approche de gestion de projet qui permet de concevoir des logiciels en impliquant, au maximum le client.

Elle est menée dans un esprit collaboratif et s'adapte aux notions incrémentales. Cette méthode engendre des produits de haute qualité tout en tenant compte de l'évolution des besoins du marché, en se basant ainsi sur la notion de communauté de projet dans laquelle les développeurs et les utilisateurs sont à la fois présents en permanence pour exprimer ou répondre à une question liée au projet[1]. Certes, elle donne l'occasion afin de gérer la qualité en continu et détecter des problèmes le plus tôt au fur et à mesure.

Le tableau 1.1 représente une comparaison des méthodologies agiles les plus populaires

TABLEAU 1.1 : Comparaison des méthodologies Agiles

Methodes	Scrum	XP	2TUP
Déscription	-Isolement de l'équipe de développement -Développement progressif -travail contrôlé quotidiennement	-Ensemble des bonnes pratiques -Cible les projets de moins de 10 personnes	-S'articule autour de l'architecture -Suit un cycle de développement en Y -Vise des projets de toutes tailles
Point forts	-Entièrement développé et testé pour courtes itérations -Simplicité des processus - Augmentation de productivité chaque équipe a son lot de responsabilité	-Itératif. -Simple à mettre en oeuvre -Fait une large place aux aspects techniques . prototypes règles de développement .	-Itératif . -Fait une large place à la technologie et à la gestion du risque . -Définit les profils d'interventions. -Apporte une réponse aux contraintes de changement continuels imposées aux systèmes informatiques .
Point faibles	-Il es bon pour les petits projets car il ne fonctionne bien qu'avec une petite équipe . -Si l'un des membres de le équipe quitte , il peut y avoir un effet inverse sur le développement du projet	-Étude de la phase d'analyse assez floue dans sa mise en oeuvre.	-N'existe pas un document type . -Trop lourd . -Exigence d'une large documentation.

1.4.1.2 Méthodologie adoptée : Scrum

Dans notre projet, nous avons choisi SCRUM comme méthodologie ,elle semblait la plus adéquate pour nos besoins, pour pleines de raisons motivantes. D'une part, il est un système léger et facile à adopter qui garantit une meilleure gestion basée sur les bonnes pratiques, et n'intervient pas dans les détails les plus techniques et spécifique de chaque projet [2]. Certes, il est distingué par son concept de rétrospective qui semble être unique à toutes les méthodes Agiles alternatives, et adopte à la fois des autres principes, au fil des sprints.

Afin de suivre l'avancement du projet, l'approche Scrum se base sur plusieurs éléments fondamentaux présentés par la figure 1.1 Ci-dessous :

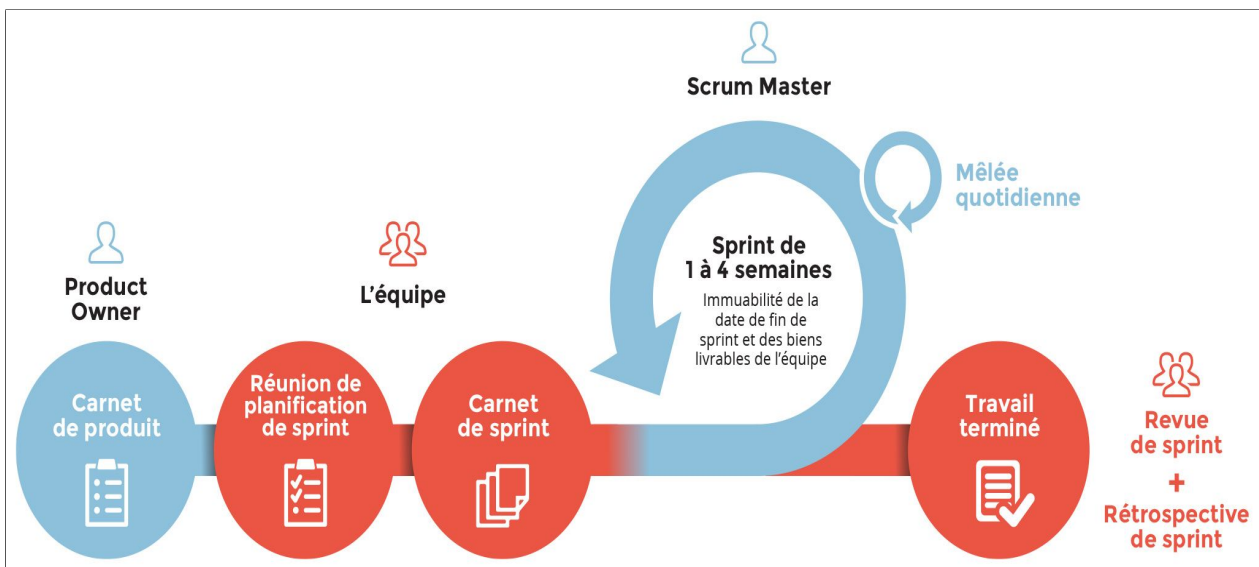


FIGURE 1.1 : La Methode Scrum Agile

— **Répartition des rôles :**

L'équipe Scrum cherche à accomplir son travail avec la plus bonne manière en se basant sur l'auto-organisation, elle se compose de :

- **Le scrum Master** qui se trouve responsable de la compréhension, de l'adhésion et de la mise en œuvre de la méthode, il doit la maîtriser parfaitement.
- **Le Product Owner** qui porte la vision du produit à réaliser, et la transfère à l'équipe de développement dont elle doit forcément suivre ses instructions.
- **L'équipe de développement** qui se charge de transformer les besoins définis par le Product Owner en fonctionnalités utilisables.

— **Les différents événements :**

Un projet Scrum se caractérise par un ensemble de réunions définies avec précision et limitées dans le temps, nous pouvons constater :

- **Le Sprint** : Une itération d'une période de 2 à 4 semaines maximum pendant laquelle une version terminée et utilisable du produit est réalisée.
- **Planification d'un Sprint** : une réunion qui permet à l'équipe d'établir et déterminer les éléments qu'elle va traiter, cette réunion est limitée à 8 heures pour les Sprints d'un mois.
- **Revue du Sprint** : une réunion de 4 heures maximum, dont l'équipe Scrum et les parties prenantes valident le bilan et évaluent ce qui a été accompli.
- **Rétrospective du Sprint** : une réunion interne qui dure 3 heures pour un Sprint d'un mois, dans le but de s'adapter aux changements qui peuvent survenir et améliorer la réalisation.
- **Mêlée quotidienne** : une réunion de 15 minutes qui se déroule à la même heure et au même endroit chaque jour, pour évaluer la progression journalière du Sprint.
- **Les artefacts** :
 - **Le product backlog** (ou carnet du produit) : une liste hiérarchisée des exigences initiales du client concernant le produit à réaliser.
 - **Le Sprint backlog** (ou carnet de Sprint) : c'est le plan d'objectif qui résume la vision de la progression.
 - **L'incrément** : l'ensemble des éléments terminés du pour le Sprint en cours, ainsi que ceux des Sprints précédents.
 - **Le Burndown Chart** (ou graphique d'avancement) : un graphique qui illustre l'avancement dans la réalisation des objectifs et tâches.

Le développement de TREND-IT InfraGen suivra une méthodologie agile, avec des itérations régulières et des phases de test pour garantir la qualité et la performance de l'application. Les principales étapes incluent :

- **Analyse et Planification** : : Identification des besoins des utilisateurs et définition des fonctionnalités clés.

- **Conception** : Création de maquettes et de prototypes de l'interface utilisateur, définition de l'architecture du backend.
- **Développement Front-end** : Implémentation de l'interface utilisateur avec Angular, intégration des fonctionnalités de glisser-déposer.
- **Développement Back-end** : Implémentation des services backend avec .NET Core, développement des API RESTful.
- **Intégration Terraform** : Développement des services d'intégration avec Terraform, génération et application des scripts.
- **Tests et Validation** : Réalisation de tests unitaires, d'intégration et utilisateur pour assurer la fiabilité et la performance de l'application.
- **Déploiement** : Mise en production de l'application, formation des utilisateurs et documentation.

1.4.1.3 Outils Scrum utilisés

Toute gestion de projet impose plusieurs actions pour atteindre les objectifs nécessaires et développer la productivité. Certes, l'acquisition d'un logiciel de gestion est très importante car elle nous permet la mise en place des tâches. D'une part, ce logiciel nous aide à la création et la planification des étapes d'une manière souple et agile, ainsi qu'indiquer pour chaque mission la valeur du travail prévu . En effet, il nous donne l'opportunité de prendre les décisions rapides et efficaces dans le but de maîtriser les risques qui nous menacent. De plus, son importance se consiste à la possibilité de suivre l'avancement de la mission en temps réel et constater sa trajectoire de déroulement. Il facilite d'une autre part la communication entre les équipes et favorise leur collaboration pour partager instantanément les informations. Nous pouvons notamment consulter nos projet même après la livraison et évaluer l'écart entre les prévisions et la réalisation. Dans ce cadre, nous avons utilisé certains outils offerts par un logiciel de gestion de projet dont nous pouvons citer :

- **Azure Devops** :

Azure DevOps est une suite de services de développement logiciel basée sur le cloud, fournie par Microsoft. Elle est conçue pour aider les équipes de développement à planifier, développer, tester et livrer des logiciels de manière plus efficace et collaborative. Azure DevOps offre un ensemble d'outils intégrés qui couvrent tout le cycle de vie du développement logiciel nous

citons Azure Boards qui fournit des outils pour la gestion de projet agile, Azure Repos et Azure pipelines.

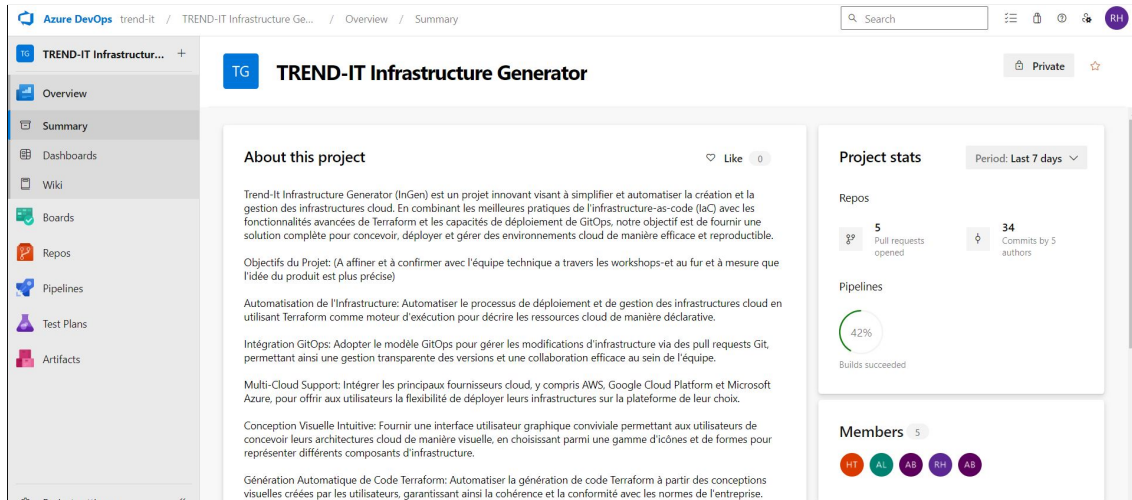


FIGURE 1.2 : Outil de Planification, développement et test : Azure Devops

— **Microsoft Teams :**

Microsoft Teams est une plateforme de communication et de collaboration développée par Microsoft. Elle est conçue pour faciliter le travail en équipe, en offrant une variété de fonctionnalités qui permettent aux utilisateurs de collaborer efficacement comme la messagerie instantanée, appels audio et vidéo, Canaux de communication et la partage des fichiers **clubhouse**. Clubhouse comprend plusieurs outils qui simplifient la communication entre les membres d'équipe.



FIGURE 1.3 : Outil de communication : Microsoft Teams

— Trello :

Trello est une application de gestion de projet et de collaboration en ligne qui utilise un système de tableaux (boards), de listes et de cartes pour organiser les tâches et les projets. Il est particulièrement apprécié pour sa simplicité et son interface visuelle intuitive

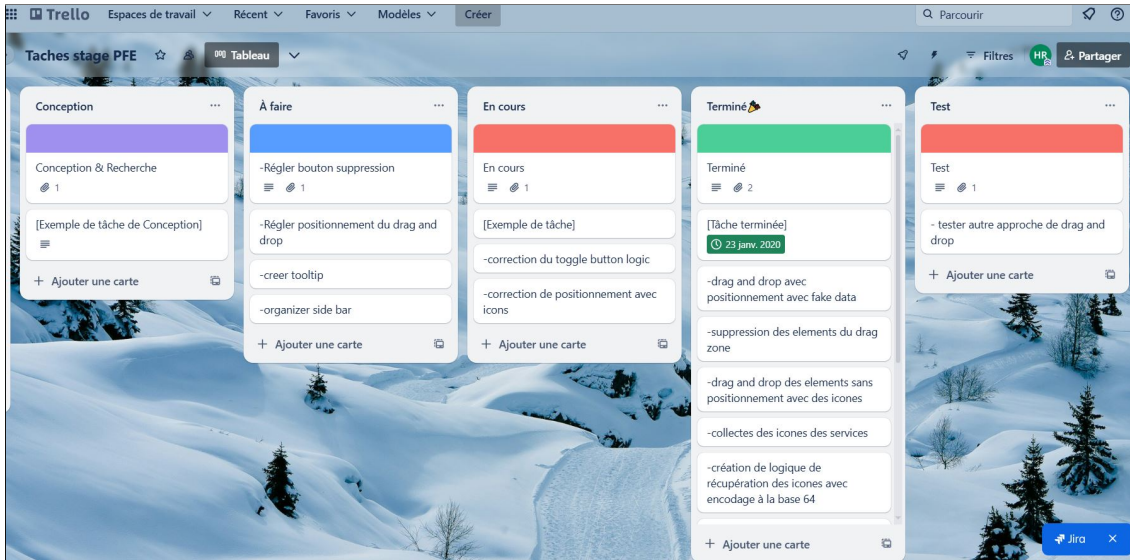


FIGURE 1.4 : plateforme de développement collaboratif : Trello

1.4.2 Formalisme adoptés

Dans ce qui suit, nous énumérons les différents diagrammes UML et les classifions en diagrammes structurels et dynamiques.

— Diagrammes structurels :

- **Diagramme de paquetage** : Les diagrammes de Paquetage décrivent l'organisation des éléments de modèle en Paquetages et les dépendances entre eux, y compris les importations de Paquetage et les extensions Paquetage . Ils fournissent également une visualisation des espaces de noms correspondants. [3]

- **Diagramme de déploiement** : Ce diagramme représente la disposition physique des ressources matérielles qui constituent le système et montre la répartition des composants (éléments logiciels) sur ces matériels. Ainsi que la nature des connexions de communication entre les différentes ressources matérielles.[4]

— Diagrammes dynamiques :

- **Diagramme de Cas d'utilisation** : Le diagramme de cas d'utilisation modélise le comportement d'un système, d'un sous-système ou d'une classe en expliquant les relations existantes entre un ensemble de cas d'utilisation et de leurs acteurs.
- **Diagramme de Séquence** : Ce diagramme décrit les interactions entre les objets en montrant la séquence temporelle des messages. Ces objets sont en général des instances de classes mais ils peuvent également représenter d'autres classificateurs tels que les acteurs
- **Diagramme d'activité** : Dans la phase de conception, les diagrammes d'activités sont particulièrement adaptés à la description des cas d'utilisation. Plus précisément, permettent de mettre l'accent sur les traitements. Ils sont donc particulièrement adaptés à la modélisation du cheminement de flots de contrôle et de flots de données. Ils permettent ainsi de représenter graphiquement le comportement d'une méthode ou le déroulement d'un cas d'utilisation[5]

Conclusion

Ce premier chapitre constitue une étape indispensable pour fixer les référentiels du projet. D'abord, nous avons présenté le contexte général et l'organisme d'accueil. Ensuite, nous avons réalisé une étude approfondie de l'existant et de critique des solutions actuelles, qui nous a permis évidemment de comprendre les besoins et envisager la solution la plus adéquate aux attentes de notre organisme de travail. De plus, nous avons déterminé grâce à ces étapes le cadre du travail, la méthodologie nécessaire, et les formalismes adoptés à emprunter lors de ce stage. Le prochain chapitre sera consacré à la présentation des besoins fonctionnels et non fonctionnels ainsi que les démarches de développement et de conception pour notre solution.

ANALYSE ET SPÉCIFICATION DES BESOINS

Plan

1	Capture des besoins globaux	17
2	Analyse des besoins fonctionnels	19
3	Pilotage du projet avec Scrum	27

CONCEPTION ET CHOIX TECHNOLOGIQUES

Plan

1	Conception architecturale de l'application	32
2	Conception détaillée	38
3	Environnements de travail et technologies	43
4	Intégration Continue et Déploiement Continu (CI/CD) :	55

3.3.2 Technologies de Développement Web

3.3.3 Angular

Angular est un framework de développement web open source, maintenu par Google, utilisé pour créer des applications web dynamiques et single-page (SPA).

— **Caractéristiques Principales :**

- **Two-Way Data Binding :** Synchronisation automatique entre le modèle et la vue.
- **Component-Based Architecture :** Facilite la réutilisation et la maintenance du code.
- **Dependency Injection :** Simplifie la gestion des dépendances entre les composants.
- **Directives :** Étendent les capacités des éléments HTML en leur attribuant des comportements spécifiques.

— **Avantages :**

- **Performance :** Gestion efficace du DOM et des données.
- **Réutilisabilité :** Composants modulaires réutilisables.
- **Communauté et Support :** Large communauté et support continu de Google.



FIGURE 3.6 : Framework Angular

3.3.3.1 .NET Core

.NET Core est un framework open source développé par Microsoft pour le développement d'applications multiplateformes.

— **Caractéristiques Principales :**

- **Cross-Platform :** Fonctionne sur Windows, Linux et macOS
- **Modularité :** Composé de modules indépendants qui peuvent être installés séparément.
- **Performance :** Performances élevées pour les applications serveur et web.
- **Unified Framework :** Utilise une base de code commune pour les applications cloud, web, IoT, et mobile.

— **Avantages :**

- **Flexibilité :** Développement pour plusieurs plateformes avec un seul code base.
- **Écosystème :** Large écosystème d'outils et de bibliothèques.
- **Sécurité :** Mises à jour régulières et robustesse des outils de sécurité intégrés.



FIGURE 3.7 : Framework.Net core

3.3.4 Outils d'Infrastructure as Code

3.3.4.1 Terraform

- **Définition :** Terraform est un outil open source développé par HashiCorp qui permet de définir et de provisionner une infrastructure informatique avec un langage de configuration simple et déclaratif.
- **Caractéristiques Principales :**
- **Déclaratif :** Les utilisateurs définissent l'état souhaité de l'infrastructure.
 - **Planification :** Génère un plan d'exécution montrant les changements qui seront appliqués.
 - **Modules :** Permet de réutiliser des configurations répétitives.
 - **Multi-Cloud :** Supporte plusieurs fournisseurs de cloud (AWS, Azure, Google Cloud, etc.).
- **Avantages :**
- **Consistance et Reproductibilité :** Assure que les mêmes configurations produisent les mêmes résultats.
 - **Gestion des Modifications :** Facilite le suivi des modifications et l'application des mises à jour.
 - **Automatisation :** Intègre facilement les workflows CI/CD pour l'automatisation des déploiements.

3.3.5 Les outils de développement utilisés

3.3.5.1 Environnements de travail

- **Teams**

Microsoft Teams a été utilisé pour les réunions, les discussions en temps réel et le partage de documents, facilitant la collaboration à distance.



FIGURE 3.8 : Microsoft Teams

- **Git**

Git est un système de contrôle de version distribué qui suit les versions des fichiers. Il est souvent utilisé pour contrôler le code source par les programmeurs développant des logiciels en collaboration.



FIGURE 3.9 : Git

- **Github**

GitHub est une plateforme de développement qui permet aux développeurs de créer, stocker, gérer et partager leur code.



FIGURE 3.10 : Github

- **Visual studio code**

Un éditeur de code source autonome qui fonctionne sous Windows, macOS et Linux. Le premier choix des développeurs JavaScript et Web, avec des extensions prenant en charge presque tous les langages de programmation.

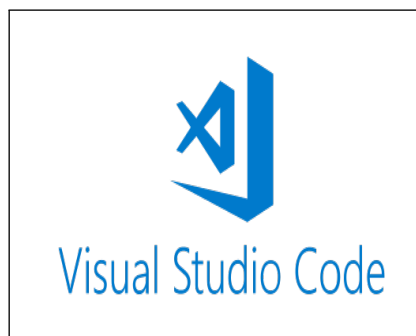


FIGURE 3.11 : Visual studio code

- **Visual Studio**

L'IDE le plus complet pour les développeurs .NET et C++ sous Windows. Entièrement doté d'une gamme complète d'outils et de fonctionnalités pour élever et améliorer chaque étape du développement logiciel.

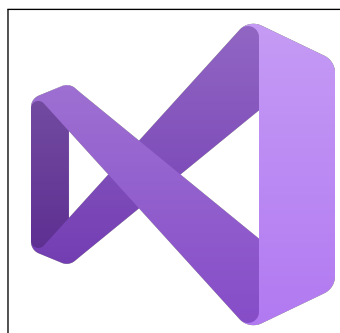


FIGURE 3.12 : Visual Studio 2022

- **Figma**

Figma a été utilisé pour le design des interfaces utilisateur, permettant de créer des prototypes interactifs et de collaborer en temps réel avec les designers.

[7]



FIGURE 3.13 : Figma

- **Draw.io**

Draw.io a été utilisé pour créer des diagrammes et des schémas techniques, facilitant la visualisation des architectures système et des flux de travail.[8]



FIGURE 3.14 : Draw.io

- **Postman**

Postman a été utilisé pour tester les API, permettant de créer et d'exécuter des requêtes HTTP pour vérifier la fonctionnalité et la performance des services web.[9]



FIGURE 3.15 : Postman

3.3.5.2 Stack Technique

Dans cette partie, nous présentons les différents frameworks et technologies choisis pour la réalisation de ce projet.

Développement frontend Pour la partie frontend, nous avons utilisé Angular V17 avec des composants standalone, Angular Material et Bootstrap.

- **Angular 16**
- **Bootstrap**

Bootstrap a été utilisé pour la conception de l'interface utilisateur, offrant des composants d'interface utilisateur modernes et réactifs pour une meilleure expérience utilisateur.

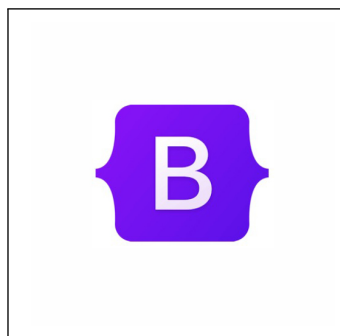


FIGURE 3.16 : Bootstrap

- **Angular Material**

Angular Material a été utilisé pour la conception de l'interface utilisateur, fournissant une bibliothèque de composants prêts à l'emploi pour Angular, facilitant le développement d'applications Web attrayantes et ergonomiques.

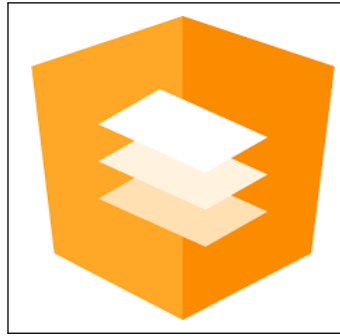


FIGURE 3.17 : Angular Material

Développement backend

- .NET Core 8**

.NET Core 8 a été utilisé pour le développement backend, permettant la création d'applications Web performantes et sécurisées avec une architecture évolutive.

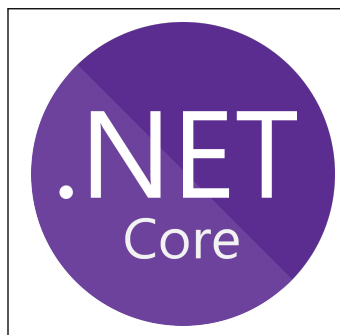


FIGURE 3.18 : .NET Core 8

- Carter**

Carter est un framework qui constitue une fine couche de méthodes d'extension et de fonctionnalités au-dessus d'ASP.NET Core. Il permet d'écrire un code plus explicite et de construire des APIs HTTP dans .NET, Cela simplifie le processus de définition et d'organisation des points d'extrémité dans une application [10].



FIGURE 3.19 : Carter

●CQRS

CQRS (Command Query Responsibility Segregation) est un modèle architectural qui sépare les opérations de lecture (requêtes) des opérations d'écriture (commandes) dans une application. Il propose de traiter ces deux aspects de manière distincte plutôt que de les gérer à travers une seule interface. [11]

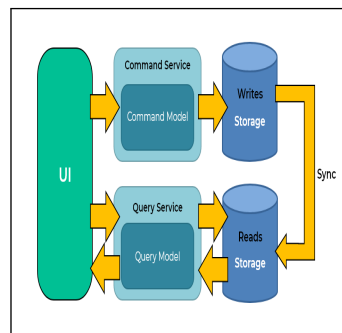


FIGURE 3.20 : CQRS

●Python

Python est un langage de programmation de haut niveau, interprété, polyvalent et dynamique. Il a été créé par Guido van Rossum et est largement utilisé dans le développement logiciel, le développement web, l'analyse de données, l'intelligence artificielle, l'automatisation des tâches, et bien plus encore.

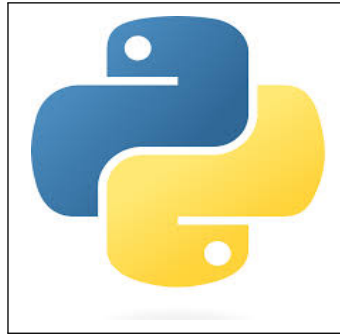


FIGURE 3.21 : .Python

- **gRPC**

gRPC (gRPC Remote Procedure Call) est un framework open source développé par Google qui facilite la communication entre les services d'application. Il est particulièrement utilisé pour construire des systèmes distribués et des microservices grâce à sa capacité à définir des API de manière concise et performante [12].



FIGURE 3.22 : gRPC

- **Terrascript**

Terrascript est une bibliothèque de génération de code pour Terraform, l'outil d'infrastructure en tant que code (IaC) développé par HashiCorp. Terraform est utilisé pour définir et fournir une infrastructure en utilisant un langage de configuration déclaratif. Terrascript permet de générer dynamiquement des configurations Terraform en utilisant un langage de programmation, comme Python, au lieu d'écrire manuellement des fichiers de configuration en HCL (HashiCorp Configuration Language).

- **Docker**

Docker a été utilisé pour containeriser les applications, facilitant le déploiement et garantissant que les applications fonctionnent de manière identique sur différents environnements.



FIGURE 3.23 : Docker

- **Docker Compose**

Docker Compose est un outil open source développé par Docker qui permet de définir et de gérer des applications multi-conteneurs. Il facilite la création et la gestion d'environnements de développement complets en utilisant Docker, en particulier lorsque plusieurs conteneurs doivent être déployés ensemble pour former une application fonctionnelle.[13]

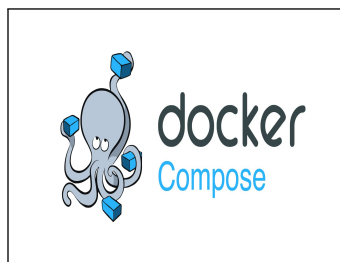


FIGURE 3.24 : Docker

- **Swagger**

La documentation Swagger est un ensemble d'outils qui permet de décrire, documenter et tester des API Web de manière structurée et cohérente. Elle facilite la compréhension et l'utilisation des API par les développeurs.

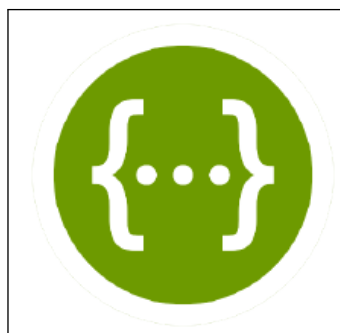


FIGURE 3.25 : Swagger

- **SQL Server Management Studio**

SQL Server Management Studio a été utilisé pour la gestion de bases de données, notamment

la création des schémas, l'exécution de requêtes et l'optimisation des performances.[14]



FIGURE 3.26 : SQL Server Management Studio

- **Terraform**

Terraform est un outil open source « infrastructure en tant que code » créé par HashiCorp qui permet aux programmeurs de créer, de modifier et de gérer les versions d'une infrastructure de manière sûre et efficace.[15]



FIGURE 3.27 : Terraform

- **Azure DevOps**

Azure DevOps prend en charge une culture collaborative et un ensemble de processus qui rassemblent les développeurs, les responsables de projets et les contributeurs pour développer des logiciels. Il permet aux organisations de créer et d'améliorer des produits à un rythme plus rapide qu'avec les approches traditionnelles de développement logiciel. [16]



FIGURE 3.28 : Azure DevOps

3.4 Intégration Continue et Déploiement Continu (CI/CD) :

CI/CD est une pratique DevOps qui combine le développement, le test et le déploiement automatisés pour accélérer les cycles de développement et améliorer la qualité des logiciels.

3.4.0.1 CI/CD Pipelines

Intégration Continue (CI) :

L'intégration continue (CI) est une pratique de développement où les développeurs intègrent régulièrement leur code dans un dépôt partagé. Chaque intégration est automatiquement vérifiée par un build automatique et des tests, permettant de détecter rapidement les erreurs. L'objectif est d'améliorer la qualité du code et de réduire le temps nécessaire pour valider et publier des mises à jour. [3]

Déploiement Continu (CD) :

Le déploiement continu (CD) est une extension de l'intégration continue qui automatise le déploiement du code validé vers des environnements de production. Cette pratique assure que les modifications de code passent automatiquement par toutes les étapes nécessaires pour atteindre la production, y compris les tests automatisés et les validations. CD permet de livrer rapidement et en toute sécurité de nouvelles fonctionnalités et corrections de bugs aux utilisateurs finaux. [3]

3.4.0.2 LE CYCLE DEVOPS

Le cycle DevOps est une combinaison des termes « développement » (Dev) et « opérations » (Ops) et se réfère à un ensemble de pratiques qui combinent le développement logiciel (développement)

et les opérations informatiques (opérations) pour raccourcir le cycle de vie du développement des systèmes et fournir une livraison continue avec une haute qualité logicielle. [4]

Les différentes étapes du cycle DevOps, comme représenté sur l'image, incluent :

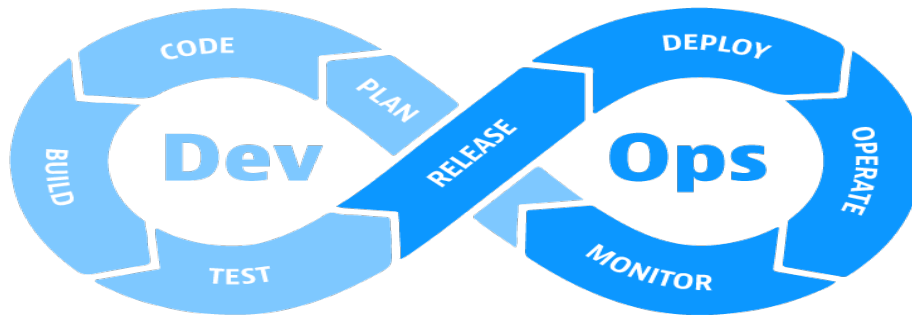


FIGURE 3.29 : Dev-Ops Logo

- **Plan** : Planification des fonctionnalités et des améliorations.
- **Code** : Écriture du code source.
- **Build** : Construction et compilation du code.
- **Test** : Test des applications pour détecter les bogues et assurer la qualité.
- **Release** : Préparation et publication de la version.
- **Deploy** : Déploiement de l'application dans l'environnement de production.
- **Operate** : Gestion et maintenance de l'application en production.
- **Monitor** : Surveillance des performances et du comportement de l'application.

Ces étapes sont représentées dans une boucle continue pour illustrer le processus itératif et continu de l'approche DevOps.

3.4.0.3 Outils CI/CD

- **Jenkins** : Serveur d'automatisation open source extensible via des plugins.
- **GitLab CI/CD** : Intégré à GitLab, fournit des pipelines CI/CD pour l'automatisation des tâches de développement.
- **Azure Pipelines** : Service CI/CD faisant partie d'Azure DevOps, permettant de créer des pipelines de build, test et déploiement.

3.4.1 Fournisseurs de Cloud :

3.4.1.1 Amazon Web Services (AWS) :

[17]

AWS est une plateforme de services cloud fournie par Amazon, offrant une large gamme de services de calcul, de stockage, de bases de données, d'analyse, de mise en réseau, de mobile, d'outils de développement, et de sécurité.

- Services Principaux :

- **EC2 (Elastic Compute Cloud)** : Instances de machines virtuelles.
- **S3 (Simple Storage Service)** : Stockage d'objets scalable.
- **RDS (Relational Database Service)** : Bases de données relationnelles gérées.
- **Lambda** : Exécution de code en réponse à des événements sans gestion de serveur.

- Avantages :

- **Évolutivité** : Facilité de mise à l'échelle des ressources.
- **Fiabilité** : Infrastructure mondiale et redondante.
- **Sécurité** : Large gamme de fonctionnalités de sécurité intégrées.

3.4.1.2 Google Cloud Platform (GCP) :

[18]

GCP est une suite de services de cloud computing proposés par Google, permettant le calcul, le stockage de données, l'analyse de données, et l'apprentissage automatique.

- Services Principaux :

- **Compute Engine** : Instances de machines virtuelles.
- **Cloud Storage** : Stockage d'objets scalable.
- **BigQuery** : Analyse de grandes quantités de données.
- **Kubernetes Engine** : Gestion des conteneurs à grande échelle

- Avantages :

- **Performance** : Infrastructure de haute performance et à faible latence

- **Intégration AI/ML** : Outils puissants pour l'intelligence artificielle et l'apprentissage automatique.
- **Tarification** : Modèle de tarification flexible et compétitif.

3.4.1.3 Microsoft Azure :

[19]

Azure est une plateforme de services cloud proposée par Microsoft, offrant une variété de services pour le calcul, l'analyse, le stockage, et les réseaux.

- Services Principaux :

- **Azure Virtual Machines** : Instances de machines virtuelles.
- **Azure Blob Storage** : Stockage d'objets scalable.
- **Azure SQL Database** : Base de données relationnelle gérée
- **Azure Functions** : Exécution de code en réponse à des événements sans gestion de serveur.

- Avantages :

- **Intégration avec l'écosystème Microsoft** : Compatibilité avec les outils et logiciels Microsoft existants.
- **Sécurité** : Large éventail de services de sécurité et de conformité.
- **MHybrid Cloud** : Capacités pour intégrer des environnements cloud et on-premise.

3.4.1.4 Etude comparative des outils

Avant de choisir l'outil d'infrastructure as code (IaC) le plus adapté à notre projet de mise en place d'une infrastructure DevOps sécurisée sur Azure avec Terraform, il est essentiel de comprendre les différences et les points forts de chaque solution. Dans cette optique, nous avons réalisé un tableau comparatif entre Terraform, Ansible et Kubernetes, en nous concentrant sur des critères clés tels que la facilité d'utilisation, la scalabilité, la flexibilité, la gestion des dépendances, l'écosystème, l'intégration avec les outils DevOps, ainsi que la maintenance et l'évolution. Cette comparaison nous aidera à prendre une décision éclairée quant à l'outil le plus approprié pour notre projet. [8]

Critères	Terraform	Ansible	AWS Cloud Formation
Facilité d'utilisation	Facile à apprendre grâce à sa syntaxe simple	Syntaxe YAML intuitive	Complexité accrue en raison de son architecture
Scalabilité	Gère efficacement les grandes infrastructures	Convient aux infrastructures de toutes tailles/problèmes de performance pour architectures complexes	limite des tailles des templates
Flexibilité	Prise en charge de nombreux fournisseurs de cloud	Gère divers types de configuration mais manque de gestion d'état	Moins flexible pour les environnements multi-Cloud
Gestion des dépendances	Gestion automatique des dépendances entre les ressources	Nécessite une gestion manuelle des dépendances	Gestion manuelle des dépendances :Template plus complexe
Écosystème	Vaste écosystème de modules et de fournisseurs	Large communauté d'utilisateurs et de modules	Fermé et Limité à la communauté AWS
Intégration avec les outils DevOps	Intégration native avec les principaux outils DevOps	Intégration via des modules et des plugins	nécessite des solutions supplémentaires pour s'intégrer
Maintenance et évolution	Gestion centralisée des configurations et des mises à jour	Mise à jour des playbooks et des rôles	Nécessite une maintenance continue

TABLEAU 3.1 : Comparaison des approches

3.4.1.5 Etude comparative des approches considérées

l'évaluation des différentes approches disponibles est cruciale pour choisir celle la plus appropriée pour générer nos scripts Terraform dans le cadre de notre projet d'infrastructure, Nous avons examiné trois méthodes pour atteindre notre objectif de manière efficace et précise. Notre analyse comparative a inclus l'utilisation de l'intelligence artificielle avec un chatbot pour automatiser la génération de scripts Terraform ou préparer un modèle, l'approche de codage manuel avec des modèles préétablis, ainsi que l'utilisation du package Terrascript pour la génération de configurations Terraform via Python. Chaque méthode a été évaluée en fonction de critères tels que la précision, la flexibilité, le coût, et le temps de développement nécessaire. Cette comparaison nous a permis de choisir Terrascript comme l'approche la plus adaptée à nos besoins actuels, offrant une combinaison de précision, de flexibilité et de facilité d'intégration dans notre flux de travail DevOps. [8]

Critères	Utilisation de l'IA avec Chatbot	Codage Manuel avec Modèles Structurés	Utilisation de Terrascript
Précision	limitations dans la création du modèle et sujette d'erreurs lors de la creations des prompts	Haute grâce à la programmation générative	Basse car sujette aux erreurs humaines.
Cout	Élevé en cas de l'intégration du chatbot	Faible mais consommateur de temps	Modéré en raison du développement et de la maintenance
Flexibilité	S'adapte aux nouvelles changements et mise à jour	Élevée grâce à la programmabilité de Python et à l'intégration facile avec d'autres outils DevOps	Limitée en raison de la rigidité des modèles.

Critères	Utilisation de l'IA avec Chatbot	Codage Manuel avec Modèles Structurés	Utilisation de Terrascript
Temps de développement	Élevé en raison de la création du modèle et moyenne en cas de l'intégration du chatbot.	Nécessite une gestion manuelle des dépendances	Gestion manuelle des dépendances entre les pods
Accessibilité aux données	Limitée en raison du besoin de données volumineuses pour entraîner le modèle.	données nécessaires accessibles	documentation accessible
Maintenance et évolution	la mise à jour régulière des prompts pour refléter les évolutions des configurations Terraform.	maintenance relativement facile grâce à la nature programmable et modulaire du code	la maintenance continue est nécessaire pour suivre les évolutions des configurations Terraform

TABLEAU 3.2 : Comparaison des solutions

Conclusion

Tout au long de ce chapitre nous avons décrit les architectures physique et logique de notre application ainsi nous avons réalisé la conception de détaillée de l'application à travers les diagrammes UML ainsi que les technologies modernes telles qu'Angular, .NET Core, Terraform et les pratiques de CI/CD permettant de créer des solutions robustes. Dans le chapitre suivant, nous aborderons la partie implémentation réelle de notre application.

REALISATION DE LA SOLUTION

Plan

1	Introduction	64
2	Déploiement de la plateforme	64
3	Réalisation des sprints	65
4	Conclusion	94
5	Perspectives	94